



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

ITERATIVE ILU PRECONDITIONERS FOR LINEAR SYSTEMS AND EIGENPROBLEMS

Item Type	Article
Authors	Boffi, Daniele; Lu, Zhongjie; Pavarino, Luca F.
Citation	sci, global. (2021). Iterative ILU Preconditioners for Linear Systems and Eigenproblems. Journal of Computational Mathematics, 39(4), 635–656. doi:10.4208/jcm.2009-m2020-0138
Eprint version	Post-print
DOI	10.4208/jcm.2009-m2020-0138
Publisher	Global Science Press
Journal	JOURNAL OF COMPUTATIONAL MATHEMATICS
Rights	Archived with thanks to JOURNAL OF COMPUTATIONAL MATHEMATICS
Download date	02/07/2023 09:11:21
Link to Item	http://hdl.handle.net/10754/670210

Iterative ILU preconditioners for linear systems and eigenproblems*

Daniele Boffi

Computer, Electrical and Mathematical Sciences and Engineering Division, King Abdullah University of Science and Technology (KAUST), Thuwal 23955-6900, Kingdom of Saudi Arabia.

Dipartimento di Matematica "F. Casorati" University of Pavia Via Ferrata 5, 27100 Pavia, Italy.

Email: daniele.boffi@unipv.it

Zhongjie Lu

School of Mathematical Sciences, University of Science and Technology of China, Hefei, Anhui 230026, China. Email: zhjlu@ustc.edu.cn

Luca F. Pavarino

Dipartimento di Matematica, Università di Pavia, via Ferrata 5, 27100 Pavia, Italy.

Email: luca.pavarino@unipv.it

Abstract

Iterative ILU factorizations are constructed, analyzed and applied as preconditioners to solve both linear systems and eigenproblems. The computational kernels of these novel Iterative ILU factorizations are sparse matrix-matrix multiplications, which are easy and efficient to implement on both serial and parallel computer architectures and can take full advantage of existing matrix-matrix multiplication codes. We also introduce level-based and threshold-based algorithms in order to enhance the accuracy of the proposed Iterative ILU factorizations. The results of several numerical experiments illustrate the efficiency of the proposed preconditioners to solve both linear systems and eigenvalue problems.

Mathematics subject classification: 65F08, 65F15, 15A23.

Key words: Iterative ILU factorization, matrix-matrix multiplication, fill-in, eigenvalue problem, parallel preconditioner

1. Introduction

The LU factorization is an efficient direct method to solve linear systems. For sparse problems, the computed L and U factors might suffer from fill-in and lose sparsity compared with the original matrix. For large-scale problems, this fact generates factors that are both time consuming to compute and memory consuming to store. In this case, iterative methods become attractive alternatives, see e.g. [6, 19, 32], since they usually require less memory and floating point computation. However, iterative methods might suffer from lack of robustness and slow convergence, and preconditioning techniques become necessary in order to make iterative methods faster and more reliable. While many preconditioners are designed for special problems, incomplete LU (ILU) preconditioners are relatively general, since they are based on the LU factorization.

There are a large number of references for the analysis and applications of ILU preconditioners in the literature of mathematics, physics and computer science, see e.g. [7, 24, 35], and ILU preconditioners are implemented in many numerical libraries [5, 26]. The convergence speed of most iterative methods depends on the distribution of the spectra of the matrices involved.

* Received xxx / Revised version received xxx / Accepted xxx /

In [8, 30], it has been proven that incomplete factorization preconditioners can essentially reduce the condition number for some matrices. There are many modifications of standard ILU based on various techniques. For example, high-level ILU(p) [32] enhances the accuracy of the factorization by allowing more fill-ins, modified ILU (MILU) [20, 27] compensates diagonal entries to make the row or column sums of the multiplication LU equal to the ones of the original matrix. Threshold ILU (ILUT(p, τ)) [31] introduces a tolerance and a number of fill-ins into the dropping rules, which provide more options to balance the accuracy and storage of the factors. There are also many references about the stability and implementation of ILUs, see e.g. [4, 16, 33].

Gaussian elimination, the core technique of ILU factorization, is a highly sequential procedure, which is an obstacle to ILUs' parallelization. Consequently, many other techniques have been introduced in order to improve the parallelism of LU factorizations. Graph coloring techniques and domain decomposition techniques have been used to partition a matrix into several submatrices, where factorizations can then be executed on each submatrix, see e.g. [21, 22, 28]. In [13], a factorization is designed based on residual correction, which can be implemented in parallel. Recently, a type of fine-grained parallel incomplete factorization based on fixed-point iterations, has been proposed in [1, 3, 14] and it has been successfully implemented in shared-memory environments.

In this paper, we construct and analyze an iterative ILU factorization of a given matrix. The main procedure of our new algorithms is based on sparse matrix-matrix multiplications. This matrix operation is nowadays contained in many computing libraries and implemented in efficient codes, which can be much faster than computing the entries one by one according to the definition. If we take full advantage of these techniques and libraries, the new algorithms are easy to implement on both share-memory and distributed-memory systems without the need to enter into the complex details of optimized serial and parallel implementations, in particular interprocessor communications. We also present two modifications of the new Iterative ILU algorithms based on high-level and threshold variants aimed at improving the efficiency.

Throughout this paper, we will not study the parallelism of the new algorithms explicitly, since the proposed preconditioners and iterative solvers require only matrix-matrix and matrix-vector multiplications, which have intrinsically fine-grained parallelism and have been implemented efficiently on different computing architectures.

The paper is organized as follows. In Section 2, we derive the conventional LU factorization in a matrix form and recall its fill-in property. We then propose several new Iterative ILU preconditioners and prove some convergence results in Section 3. In Section 4, we discuss some issues involved in the new algorithms, such as matrix-matrix multiplication and solving triangular systems. In Section 5 and 6, we apply the new preconditioner to solving linear systems and eigenvalue problems, respectively, showing the results of several numerical experiments. Some conclusions and limitations are reported in Section 7.

2. Iterative LU factorization

Let A be a square matrix that can be factorized as a multiplication of a lower and an upper triangular matrices, i.e.

$$A = LU, \tag{2.1}$$

where all the diagonal entries of L are 1. We write the triangular matrices L and U in the following forms:

$$L = L_0 + I \quad \text{and} \quad U = U_0 + D,$$

where I is the identity matrix and D is the diagonal matrix of U . We rewrite (2.1) as

$$A = (L_0 + I)(D + U_0) = L_0D + D + U_0 + L_0U_0$$

or

$$L_0D + D + U_0 = A - L_0U_0. \quad (2.2)$$

Based on the equation (2.2), we obtain an iterative form for LU factorization:

$$L_0^k D^k + D^k + U_0^k = A - L_0^{k-1} U_0^{k-1}, \quad (2.3)$$

where $L_0^k D^k$, D^k and U_0^k are strictly lower triangular, diagonal and strictly upper triangular matrix of the matrix $A - L_0^{k-1} U_0^{k-1}$, respectively, and L_0^k is easy to obtain from $L_0^k D^k$. In this paper, we assume that all the diagonal entries of D^k ($k \geq 1$) are not zero.

For a square matrix B , we use Matlab-like notations $\text{diag}(\text{diag}(B))$, $\text{tril}(B, -1)$ and $\text{triu}(B, 1)$ to denote the diagonal, strictly lower triangular and strictly upper triangular part of B , respectively. This iterative form (2.3) can be implemented as Algorithm 2.1.

Algorithm 2.1 Iterative LU factorization

- 1: Set the initial data $L_0 = D = U_0 = 0$.
 - 2: **for** $k = 1, 2, \dots, p$ **do**
 - 3: $B = A - L_0 U_0$
 - 4: $D = \text{diag}(\text{diag}(B))$
 - 5: $U_0 = \text{triu}(B, 1)$
 - 6: $L_0 = \text{tril}(B, -1) D^{-1}$
 - 7: **end for**
 - 8: $L = L_0 + I$ and $U = U_0 + D$.
-

The convergence properties of this algorithm are analyzed in the following theorem. Since it is a special case of Theorem 3.1, we will discuss it after proving Theorem 3.1 in Section 3.

Theorem 2.1. *The factors L, U in Algorithm 2.1 converge to the exact factorization of A within at most n steps.*

2.1. Fill-in of the factors

Let S_A be the set of index pairs (i, j) , $i, j \leq n$ associated to the sparsity pattern of the matrix A . For large sparse matrices, we only allocate memory for the entries in a sparsity pattern. We define the sparsity pattern of the sum and product of two matrices L and U :

$$S_{L+U} = \{(i, j) \mid (i, j) \in S_L \text{ or } (i, j) \in S_U\} = S_L \cup S_U, \quad (2.4)$$

$$S_{LU} = \{(i, j) \mid \exists k, \text{ s.t. } (i, k) \in S_L \text{ and } (k, j) \in S_U\}. \quad (2.5)$$

The number of nonzeros of the factors increases with the increasing iteration number k , as it involves the matrix-matrix multiplication $L_0 U_0$ in step 3 of in Algorithm 2.1. We call the new nonzeros in the factors compared with the original matrix A *fill-ins*. To study the fill-in property of the factors, we introduce the concepts of *fill-path* and *level-of-fill* in sparse direct solution methods [32]. If a matrix is regarded as a oriented graph, a *fill-path* of two vertexes i and j is a path between them, such that all the vertexes in this path, except the end points i and j , are numbered less than i and j . If the length of the shortest path between i and j is $p + 1$, the *level-of-fill* value of the position (i, j) is p . If the level-of-fills of all the positions in a sparsity pattern of a matrix are less than or equal to p , we call this pattern *level- p* pattern of this matrix. Then the level-of-fills of the nonzero positions of a matrix are 0, and the sparsity pattern consisting of all the nonzero positions is a level-0 pattern of this matrix. Also, there is the following result about the level-1 pattern.

Lemma 2.1. [3, 32] *If the sparsity patterns of L and U are the same as the lower and upper part of A , respectively, then the sparsity pattern of the product LU is the level-1 pattern of A .*

Theorem 2.2. *For $k \geq 2$, the sparsity pattern of $L_0^k + D^k + U_0^k$ is the level-1 pattern of $L_0^{k-1} + D^{k-1} + U_0^{k-1}$.*

Proof. In the k^{th} ($k \geq 1$) iteration, by (2.4) and (2.5) $B^k = A - L_0^{k-1} U_0^{k-1}$ (step 3) implies

$$S_{B^k} = S_A \cup S_{L_0^{k-1} U_0^{k-1}} \quad (2.6)$$

and $L_0^k D^k + D^k + U_0^k = B^k$ (step 4, 6 and 5) implies

$$S_{B^k} = S_{L_0^k} \cup S_{D^k} \cup S_{U_0^k} \quad (2.7)$$

Also, by (2.4), (2.5) and (2.7), we have an expansion

$$\begin{aligned} S_{L^k U^k} &= S_{(L_0^k + I)(D^k + U_0^k)} \\ &= S_{L_0^k D^k} \cup S_{D^k} \cup S_{U_0^k} \cup S_{L_0^k U_0^k} \\ &= S_{L_0^k} \cup S_{D^k} \cup S_{U_0^k} \cup S_{L_0^k U_0^k} \\ &= S_{B^k} \cup S_{L_0^k U_0^k}. \end{aligned} \quad (2.8)$$

By Lemma 2.1, the conclusion of this theorem is equivalent to

$$S_{B^k} = S_{L^{k-1} U^{k-1}}.$$

In the following, we prove this by induction.

For $k = 1$, we have $B = A$ as the initial data. Then by (2.8), we have

$$S_{L^1 U^1} = S_B \cup S_{L_0^1 U_0^1} \equiv S_A \cup S_{L_0^1 U_0^1} \quad (2.9)$$

For $k = 2$, from step 3 and (2.9), we have

$$S_B = S_A \cup S_{L_0^1 U_0^1} = S_{L^1 U^1}.$$

For $k \geq 3$, we assume by induction that

$$S_{B^k} = S_{L^{k-1} U^{k-1}}.$$

By (2.7) and (2.8), we expand its both sides

$$\begin{aligned} S_{L_0^k} \cup S_{D^k} \cup S_{U_0^k} &= S_{B^k} = S_{L^{k-1}U^{k-1}} \\ &= S_{L_0^{k-1}} \cup S_{D^{k-1}} \cup S_{L_0^{k-1}} \cup S_{L_0^{k-1}U_0^{k-1}}. \end{aligned}$$

which implies

$$S_{L_0^k} \supset S_{L_0^{k-1}} \quad \text{and} \quad S_{U_0^k} \supset S_{U_0^{k-1}}.$$

and consequently

$$S_{L_0^k U_0^k} \supset S_{L_0^{k-1} U_0^{k-1}}. \quad (2.10)$$

Then for the next iteration $k+1$, we have

$$\begin{aligned} S_{B^{k+1}} &= S_A \cup S_{L_0^k U_0^k} \\ &= S_A \cup S_{L_0^{k-1} U_0^{k-1}} \cup S_{L_0^k U_0^k} \quad (\text{using (2.10)}) \\ &= S_{B^k} \cup S_{L_0^k U_0^k} \quad (\text{using (2.6)}) \\ &= S_{L^k U^k}, \quad (\text{using (2.8)}) \end{aligned}$$

which proves the claim for the index $k+1$.

Remark 2.1. In Algorithm 2.1, the sparsity pattern of the factorizations (L^1, U^1) and (L^2, U^2) are the level-0 and level-1 patterns of the original matrix. However, it is a progressive relation in Theorem 2.2, which is different from the conventional concept of level- $(k-1)$ pattern, when $k \geq 3$. Usually, in k^{th} ($k \geq 3$) iteration, the pattern (L^k, U^k) is larger than the level- $(k-1)$ pattern.

3. Iterative incomplete LU

The number of fill-ins in the factors of Algorithm 2.1 increases rapidly when the number of iterations increases, because of the matrix-matrix multiplication $L_0 U_0$ in step 3. In order to control the storage of the factors, we drop some entries in each iteration according to some rules and stop the iteration with an appropriate number, which is the reason that we call it the incomplete LU (ILU) factorization. We summarize this idea in Algorithm 3.1. In the following of this section, we introduce two dropping strategies, based on pattern and threshold, respectively.

3.1. Pattern-based

In Algorithm 3.2, we set a sparsity pattern $\mathbf{S}$ in advance, and drop the entries of L_0 and U_0 out of $\mathbf{S}$ in each iteration. If $\mathbf{S}$ contains the sparsity pattern of A , after the first iteration, $L_0 = A_{L0} A_D^{-1}$ and $U_0 = A_{U0}$, where A_{L0} , A_D and A_{U0} are the strictly lower triangular, diagonal and strictly upper triangular parts of the original matrix A , respectively. Then, the corresponding ILU preconditioner $LU = (A_{L0} + A_D) A_D^{-1} (A_D + A_{U0})$ is the SSOR(1) preconditioner [32], which can be regarded as the de facto initial data of this algorithm. This is a reasonable initial guess, as the SSOR(1) preconditioner already works well for some problems.

In the following, we prove the convergence of Algorithm 3.2. We use Figure 3.1 to illustrate the procedure of the proof.

Algorithm 3.1 Iterative ILU factorization

```

1: Set the initial data  $L_0 = D = U_0 = 0$ .
2: for  $k = 1, 2, \dots, p$  do
3:    $B = L_0 U_0$ 
4:    $B = A - B$ 
5:    $D = \text{diag}(\text{diag}(B))$ 
6:    $U_0 = \text{triu}(B, 1)$ 
7:    $L_0 = \text{tril}(B, -1)D^{-1}$ 
8:   execute dropping strategies on  $L_0$  and  $U_0$ .
9: end for
10:  $L = L_0 + I$  and  $U = U_0 + D$ .

```

Algorithm 3.2 Iterative ILU factorization

```

1: Set the initial data  $L_0 = D = U_0 = 0$  and the sparsity pattern  $\mathbf{S}$ .
2: for  $k = 1, 2, \dots, p$  do
3:    $B = L_0 U_0$ 
4:    $B = A - B$ 
5:    $D = \text{diag}(\text{diag}(B))$ 
6:    $U_0 = \text{triu}(B, 1)$ 
7:    $L_0 = \text{tril}(B, -1)D^{-1}$ 
8:   drop the entries of  $L_0$  and  $U_0$  out of the pattern  $\mathbf{S}$ .
9: end for
10:  $L = L_0 + I$  and  $U = U_0 + D$ .

```

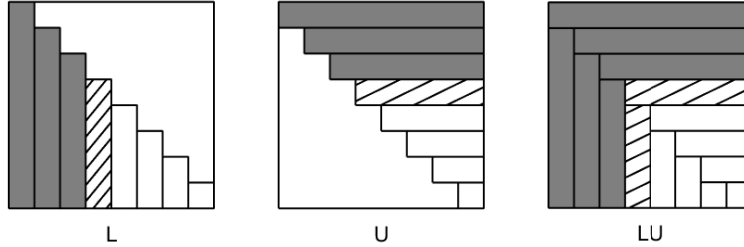


Fig. 3.1.: The entries in the part marked by back shadow in each matrix are exact values. In the next iteration, these exact entries will not change (Lemma 3.1), and the rows and columns marked by slash shadow will become exact (Lemma 3.2).

Lemma 3.1. *In Algorithm 3.2, if the product LU is equal to A on the first k rows and the first k columns of the sparsity pattern $\mathbf{S}$, then the first k columns of L and first k rows of U do not change in the next iteration.*

Proof. As the dropping rule in step 8, we only consider the entries in the pattern $\mathbf{S}$ during the proof. Let l_{ij} and u_{ij} denote the entries of the lower and upper triangular factors L and

U , respectively. As in the settings of Section 2, all the diagonal entries are $l_{ii} = 1$, and the diagonal entries of D and U are the same, i.e. $u_{ii} = d_{ii}$. As L_0 and U_0 are the strictly lower and upper parts of L and U , respectively, we still use l_{ij} and u_{ij} to denote the nonzero entries in L_0 and U_0 . Let l'_{ij} and u'_{ij} be the updated values of l_{ij} and u_{ij} in the next iteration, respectively. From the assumption, we have (see the black part in the third figure in Figure 3.1)

$$a_{ij} = \sum_{m=1}^{\min(i,j)} l_{im}u_{mj} \quad \text{for } i \leq k \text{ or } j \leq k, \quad (i,j) \in \mathbf{S},$$

and divide the sum into two parts:

$$\begin{aligned} a_{ij} &= \sum_{m=1}^{i-1} l_{im}u_{mj} + u_{ij} \quad \text{for } i \leq k, i \leq j, \quad (i,j) \in \mathbf{S}, \\ a_{ij} &= \sum_{m=1}^{j-1} l_{im}u_{mj} + l_{ij}u_{jj} \quad \text{for } j \leq k, i > j, \quad (i,j) \in \mathbf{S}, \end{aligned} \quad (3.1)$$

(here we use the setting $l_{ii} = 1$).

In the next iteration,

$$b_{ij} = a_{ij} - \sum_{m=1}^{\min(i-1,j-1)} l_{im}u_{mj}. \quad (\text{step 3 and 4}) \quad (3.2)$$

The new upper factor becomes

$$u'_{ij} = b_{ij} = a_{ij} - \sum_{m=1}^{i-1} l_{im}u_{mj} \quad \text{for } i \leq k, i \leq j, \quad (\text{step 6})$$

and by the first equation of (3.1), we have

$$u'_{ij} = u_{ij}, \quad \text{for } i \leq k. \quad (3.3)$$

The new lower factor becomes

$$l'_{ij}u'_{jj} = b_{ij} = a_{ij} - \sum_{m=1}^{j-1} l_{im}u_{mj} \quad \text{for } j \leq k, i > j, \quad (\text{step 5 and 7})$$

and by the second equation of (3.1), we have $l'_{ij}u'_{jj} = l_{ij}u_{jj}$. From (3.3), we obtain $l'_{ij} = l_{ij}$ for $j \leq k$.

Lemma 3.2. *In Algorithm 3.2, if the product LU is equal to A on the first k ($k \leq n-1$) rows and the first k columns of the sparsity pattern $\mathbf{S}$, then in the next iteration, LU is equal to A on the first $k+1$ rows and the first $k+1$ columns of the sparsity pattern $\mathbf{S}$.*

Proof. By Lemma 3.1, we only need to investigate the $(k+1)^{th}$ row and $(k+1)^{th}$ column of the new factors.

From (3.2), for $j = k+1, \dots, n$ and $(k+1, j) \in \mathbf{S}$,

$$u'_{k+1,j} = b_{k+1,j} = a_{k+1,j} - \sum_{m=1}^k l_{k+1,m}u_{mj}.$$

We then have

$$a_{k+1,j} = \sum_{m=1}^k l'_{k+1,m} u'_{m,j} + l'_{k+1,k+1} u'_{k+1,j} = \sum_{m=1}^n l'_{k+1,m} u'_{m,j}.$$

Here we use the setting $l'_{k+1,k+1} = 1$ and the result of Lemma 3.1, i.e. $l_{k+1,m} = l'_{k+1,m}$ and $u_{m,j} = u'_{m,j}$ because of $m \leq k$. Then the $(k+1)^{th}$ row of U , i.e. $u_{k+1,j}$ for $j = k+1, \dots, n$, become exact values (see the yellow part of the second panel in Figure 3.1).

Similarly, for $i = k+2, \dots, n$ and $(i, k+1) \in \mathcal{S}$, we have

$$l'_{i,k+1} u'_{k+1,k+1} = b_{i,k+1} = a_{i,k+1} - \sum_{m=1}^k l_{i,m} u_{m,k+1},$$

$$a_{i,k+1} = \sum_{m=1}^k l'_{i,m} u'_{m,k+1} + l'_{i,k+1} u'_{k+1,k+1} = \sum_{m=1}^n l'_{i,m} u'_{m,k+1}.$$

Then the $(k+1)^{th}$ column of L , i.e. $l_{i,k+1}$ for $i = k+2, \dots, n$, become exact values (see the yellow part of the first panel in Figure 3.1).

Theorem 3.1. *The factors in Algorithm 3.2 converge to the exact incomplete factorization on the sparsity pattern $\mathcal{S}$ within n iterations.*

Proof. After the iteration $k = 1$, the entries in the first column of L^1 and the first row of U^1 are, respectively,

$$l_{i1}^1 = \begin{cases} a_{i1}/a_{11} & (i, 1) \in \mathcal{S} \\ 0 & (i, 1) \notin \mathcal{S} \end{cases} \quad \text{and} \quad u_{1j}^1 = \begin{cases} a_{1j} & (1, j) \in \mathcal{S} \\ 0 & (1, j) \notin \mathcal{S}. \end{cases} \quad (3.4)$$

Then it can be verified that the multiplication $L^1 U^1$ is equal to A on the first row and the first column of the sparsity pattern $\mathcal{S}$.

By Lemma 3.1 and 3.2, we know that one more iteration makes one more row and column exact until we obtain the exact ILU factorization on $\mathcal{S}$.

Theorem 2.1 is a special case of Theorem 3.1, when $\mathcal{S}$ contains all the positions of the matrix A .

The Algorithm 3.2 also has the following symmetry preserving property.

Theorem 3.2. *If A is symmetric and $\mathcal{S}$ is a symmetric pattern, the preconditioner generated by Algorithm 3.2 preserves the symmetry, i.e. the product LU is symmetric.*

Proof. As $LU = (L_0 + I)(U_0 + D) = L_0 D + D + U_0 + L_0 U_0$, it suffices to prove that $L_0 = U_0^T D^{-1}$ at step 8 of Algorithm 3.2.

For $k = 1$, we have the following results in each step of Algorithm 3.2:

- step 3: $B = 0$ is symmetric.
- step 4: $B = A$ is symmetric.
- step 5-7: We have $L_0 = U_0^T D^{-1}$.
- step 8: As $\mathcal{S}$ is symmetric, the dropped elements in L_0 and U_0 are correspondingly symmetric. Then we still have $L_0 = U_0^T D^{-1}$.

For $k \geq 2$, we assume that $L_0 = U_0^T D^{-1}$ in the previous step. Then we have

- step 3: $B = L_0 U_0 = U_0^T D^{-1} U_0$ is symmetric. (Here, L_0 and U_0 are given in the previous iteration.)
- step 4: $B = A - B$ is symmetric.
- step 5-7: We have $L_0 = U_0^T D^{-1}$. (Here, L_0 and U_0 are updated in the current iteration.)
- step 8: As $\mathbf{S}$ is symmetric, we still have $L_0 = U_0^T D^{-1}$.

The level-0 sparsity pattern is the most commonly used in ILU factorizations. To obtain more accurate factors, we can enlarge the sparsity pattern to allow more fill-ins. If we iterate Algorithm 3.2 several times without dropping any entries, the resulting factors are of a larger sparsity pattern by Theorem 2.2 and Remark 2.1. Then the factors and their sparsity pattern can be taken as the initial data and a fixed pattern, respectively, when using Algorithm 3.2. We summarize this strategy in Algorithm 3.3, and call it $\text{IterILU}(p, m)$. Here p indicates the size of the pattern and m is the iteration number to enhance the accuracy of the factors on this pattern. We observe that Algorithms 2.1 and 3.2 can be regarded as special cases of Algorithm 3.3:

- $p = 1$ and $m = 0$, SSOR(1) preconditioner;
- $p = 1$ and $m \geq 1$, Algorithm 3.2 with m iterations on level-0 sparsity pattern;
- $p \geq 2$ and $m = 0$, Algorithm 2.1 with p iterations;
- $p \geq 2$ and $m \geq 1$, Algorithm 2.1 with p iterations, then on the sparsity pattern of the results, execute Algorithm 3.2 with m enhancement iterations.

Usually, the storage increases rapidly with increasing p . When setting p , the memory in the computing resource and the time for solving the corresponding triangular systems should be taken into consideration. We write a short code for this algorithm in Matlab, pasted in Appendix A.

3.2. Threshold-based

We propose another dropping strategy for L_0 and U_0 , which is based on only the magnitudes of the entries in each row or column:

- Find the largest absolute value Γ in each row of L , delete the entries whose absolute values are smaller than $\tau\Gamma$ in the corresponding row of L_0 .
- Find the largest absolute value Γ in each column of U , delete the entries whose absolute values are smaller than $\tau\Gamma$ in the corresponding column of U_0 .

Here, if A is symmetric and the threshold τ is uniformly set for L_0 and U_0 , the symmetry property of the preconditioner is preserved. We call this algorithm $\text{IterILUT}(\tau, p)$ in Algorithm 3.4, where p is the iteration number.

The application range of the threshold-based factorization may be smaller than the pattern-based ones. In some matrices, the magnitudes of their entries may cluster in some small regions, not relatively uniform distribute on the number axis. For example, the matrices generated by

Algorithm 3.3 Iterative ILU factorization IterILU(p, m)

```

1: Set the initial data  $L_0 = D = U_0 = 0$ .
2: for  $k = 1, 2, \dots, p$  do
3:    $B = A - L_0 U_0$ 
4:    $D = \text{diag}(\text{diag}(B))$ 
5:    $U_0 = \text{triu}(B, 1)$ 
6:    $L_0 = \text{tril}(B, -1)D^{-1}$ 
7: end for
8: get the sparsity pattern  $\mathcal{S}$  of  $B$ 
9: for  $k = 1, 2, \dots, m$  do
10:   $B = L_0 U_0$ 
11:   $B = A - B$ 
12:   $D = \text{diag}(\text{diag}(B))$ 
13:   $U_0 = \text{triu}(B, 1)$ 
14:   $L_0 = \text{tril}(B, -1)D^{-1}$ 
15:  drop the entries of  $L_0$  and  $U_0$  out of the pattern  $\mathcal{S}$ .
16: end for
17:  $L = L_0 + I$  and  $U = U_0 + D$ .

```

Algorithm 3.4 Iterative threshold ILU factorization IterILUT(τ, p)

```

1: Set the initial data  $L_0 = D = U_0 = 0$ .
2: for for  $k = 1, 2, \dots$ , do
3:    $B = L_0 U_0$ 
4:    $B = A - B$ 
5:    $D = \text{diag}(\text{diag}(B))$ 
6:    $U_0 = \text{triu}(B, 1)$ 
7:    $L_0 = \text{tril}(B, -1)D^{-1}$ 
8:   execute dropping strategies on  $L_0$  and  $U_0$  according to the threshold  $\tau$ .
9: end for
10:  $L = L_0 + I$  and  $U = U_0 + D$ .

```

FDM or FEM on uniform meshes contain many entries with the same values. If the threshold τ is too large, an important part of the matrix may be dropped, which heavily damages the accuracy of the factors. If τ is too small, there may be no effect in reducing the storage. This strategy should be applied to those matrices whose entries distribute smoothly. And the threshold also should be chosen deliberately according to the desired accuracy and storage of the factors. Of course, there should be hybrid dropping strategies based on positions, numbers and magnitudes of entries.

4. Some other issues involved in the algorithms

In this section, we discuss some details in the implementation of the new algorithms.

4.1. Matrix-matrix multiplication

The main computational task of the new algorithms is the sparse matrix-matrix multiplication which is a fundamental operation in the field of high-performance computing. The existing algorithms and codes for this operation have been studied by many experts in various aspects for many years [12, 18, 29]. Many theories and techniques have been introduced for the optimization of its algorithms, especially for sparse cases. Most computing libraries contain efficient codes for this operation on both sequential and parallel computer architectures.

A risk in performing this operation is that the product may contain too many fill-ins, even though the two matrices are very sparse. In some extreme cases, it can even result in a full matrix, for example, the first column of L_0 and the first row of U_0 are nonzero. The reordering techniques can reduce the number of fill-ins for some cases [23], and also possibly improve the accuracy of the factors. Another method is that we only compute the entries contained in the given pattern during the matrix-matrix multiplication, which we call **the Incomplete Matrix-Matrix Multiplication** (IMMM). Algorithm 4.1 implements this idea, which is mathematically equivalent to, but theoretically cheaper than Algorithm 3.2. However, to the authors'

Algorithm 4.1 Iterative ILU factorization

```

1: Set the initial data  $L_0 = D = U_0 = 0$ .
2: for  $k = 1, 2, \dots, p$  do
3:    $B = L_0 U_0$  (IMMM on  $\mathcal{S}$ )
4:    $B = A - B$  (on  $\mathcal{S}$ )
5:    $D = \text{diag}(\text{diag}(B))$ 
6:    $U_0 = \text{triu}(B, 1)$ 
7:    $L_0 = \text{tril}(B, -1)D^{-1}$ 
8: end for
9:  $L = L_0 + I$  and  $U = U_0 + D$ .
```

knowledge, there is no such operation in the existing libraries. It is easy to implement in parallel according to the definition on shared-memory platforms, while on distributed-memory platforms, the users have to deal with the communications between different processors. If the memory and runtime can afford these fill-ins, Algorithm 3.2 and 3.3 are recommended.

4.2. Triangular system solvers

After obtaining the factors of an incomplete factorization, the preconditioning procedure requires solving two triangular linear systems $L(Uy) = x$. The forward or backward substitution is a common method, which can be parallelized by level scheduling [2]. Another method is to use Jacobi iterations [3]. In the iteration schemes:

$$\begin{aligned} z^k &= x - L_0 z^{k-1}, \\ y^k &= D^{-1} z - D^{-1} U_0 y^{k-1}, \end{aligned} \tag{4.1}$$

the iteration matrices $-D^{-1}U_0$ and $-L_0$ are strictly upper and lower triangular matrix. We summarize this method in Algorithm 4.2. The spectral radii of the two matrices are both 0, which means fast convergence of the schemes. By using this method, the main task of the preconditioned iterative solver remains only matrix-vector products, which benefits from the

parallel implementation. Moreover, the preconditioned solution is only a quite rough approximation for the exact solution, so there is no need to iterate the schemes (4.1) until reaching a very small error. In general, a few iteration steps for each scheme are accurate enough for preconditioning purposes.

Algorithm 4.2 Jacobi method for the systems $L(Uy) = x$ with p iterations.

```

1: Set the initial data  $z^0 = 0$  and  $y^0 = 0$ .
2: for for  $k = 1, 2, \dots, p$  do
3:    $z^k = x - L_0 z^{k-1}$ 
4: end for
5: for for  $k = 1, 2, \dots, p$  do
6:    $y^k = D^{-1} z^p - D^{-1} U_0 y^{k-1}$ 
7: end for
8: output  $y = y^p$ 

```

5. Numerical experiments I: linear systems

In this section, we apply the new preconditioners to solving linear systems with the Preconditioned Conjugate Gradient method (PCG). The main purpose is to determine the parameters in the new algorithms and to compare the new preconditioners with the standard ILU.

The code is implemented in Matlab and run on a laptop with an Intel i7-6700HQ CPU and 16 GB RAM.

5.1. IterILU(p, m)

In this section, we consider linear system with coefficient matrices originating from finite difference discretizations of the Laplacian with homogeneous Dirichlet boundary condition and we investigate the basic properties and effects of the iterative ILU preconditioners IterILU(p, m) in Algorithm 3.3. The two- and three-dimensional grids are 102×102 and $102 \times 102 \times 102$, so the corresponding square matrices have 10^4 and 10^6 rows, respectively.

Firstly, we test the fill-in property of the factors generated by IterILU(p, m). Table 5.1 shows the numbers of nonzeros of L for different p , and the ratios between these number and the number of nonzeros of the case $p = 1$. The numbers of nonzeros increase with p and the 3D ratios increase more rapidly than the 2D ratios, as its matrices have more nonzero entries in each row or column. In order to keep the storage required by the factors under control, we restrict our tests to the values $p = 1, 2, 3$.

The next two tests focus instead on the effects of varying the enhancement number m . The test matrix is the 3D Laplacian. In order to show the difference between incomplete factors and the exact one, we introduce a relative error:

$$\text{relative error}(A, LU) = \max_i \left\{ \frac{\sum_j^n |A_{ij} - (LU)_{ij}|}{\sum_j^n |A_{ij}|} \right\}. \quad (5.1)$$

Figure 5.1 (left) shows that the relative errors decrease with increasing m , but reach a plateau after a few enhancement iterations m . We then employ the factors of IterILU(p, k) as preconditioners within the PCG method with right-hand side given by random values in $[-1, 1]$

	2D Laplace 100×100		3D Laplace $100 \times 100 \times 100$	
	nnz	ratio	nnz	ratio
$p = 1$	29800	1	3970000	1
$p = 2$	39601	1.33	6910300	1.74
$p = 3$	49303	1.65	12721996	3.20
$p = 4$	68608	2.30	28972351	7.30
$p = 5$	97025	3.26	72694564	18.31
$p = 6$	143276	4.81	201462286	50.75

Table 5.1.: Numbers of nonzeros in the lower triangular factor L generated by $\text{IterILU}(p, m)$ for increasing p and $m = 0$ for the 2D and 3D finite difference Laplacian.

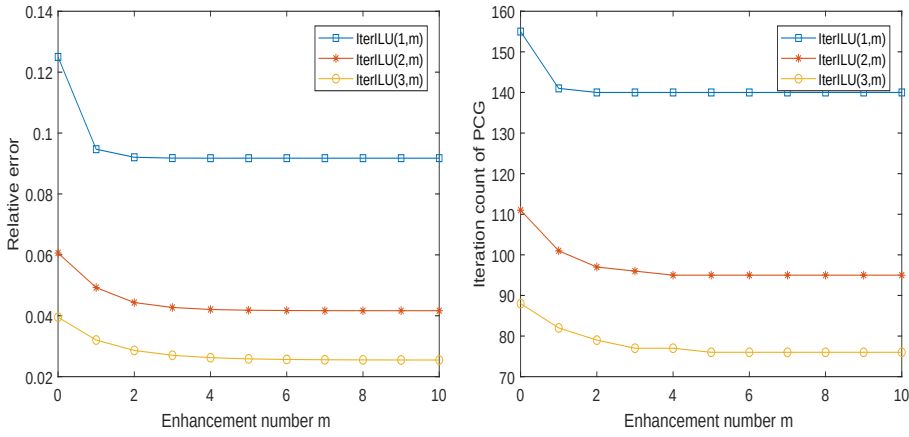


Fig. 5.1.: Left: relative errors of $\text{IterILU}(p, m)$ with $p = 1, 2, 3$ and different enhancement numbers m . Right: PCG iteration counts with different values of p and m .

and 0 initial gauss. The PCG iteration numbers reach a stable value after a few enhancement iterations, shown in Figure 5.1 (right). From the two panels of Fig. 5.1, we conclude that 3 enhancement iterations are enough for both the relative errors and iteration numbers to reach stable values.

In the test above, the triangular systems $L(Uy) = x$ in the preconditioning procedures of PCG are solved using a direct method. In the next numerical experiment, we use instead the Jacobi iterative method (4.1), Algorithm 4.2 to solve the two triangular systems. Figure 5.2 shows the relation between the PCG and Jacobi iteration counts. From the results, we observe that the PCG iteration counts with $\text{IterILU}(p, m)$ preconditioners become stable after 6 iterations for $(p, m) = (1, 3)$, 8 iterations for $(p, m) = (2, 3)$ and 12 iterations for $(p, m) = (3, 3)$. Hence, it is not appropriate to set a uniform value for the iteration number q in the Jacobi method. If q is too small, the accuracy of preconditioning may be not enough for some problems, while if it is too large, it is possible to waste computation. This number should be calibrated according to the specific problem considered. In order to eliminate the influence of different settings for this number, we use an exact solver for the triangular systems in the remaining tests.

We plot the PCG convergence history using $\text{IterILU}(p, m)$ with different parameters and compare them with the exact $\text{ILU}(0)$ preconditioner, Figure 5.3. $\text{IterILU}(1, 3)$ and exact $\text{ILU}(0)$

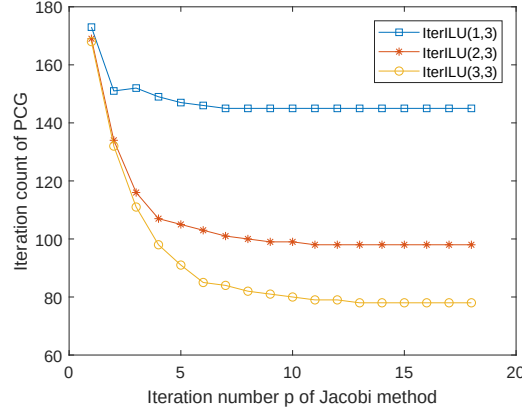


Fig. 5.2.: PCG iteration counts using the Jacobi method of Algorithm 4.2 with different iteration numbers for the preconditioner triangular solves.

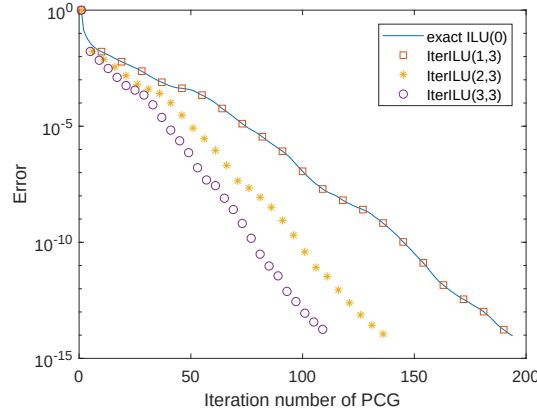


Fig. 5.3.: Convergence history for solving the 3D Laplace problem using PCG with different ILU preconditioners.

yield the same results, which means they have the similar effects for this problem. The differences are that IterILU(1,3) is easier to generate and has naturally fine-grained parallelism. IterILU(2,3) and IterILU(3,3) yield faster PCG convergence than the level-0 case, at the cost of allowing more entries in their triangular factors.

Next, we carry out the same study for some symmetric matrices from the University of Florida sparse matrix collection [15]. Table 5.2 lists the dimension, number of nonzero entries and ratios of numbers of nonzeros in the IterILU(p, m) factors generated with different p . Table 5.3 shows the iteration counts of PCG using different preconditioners. For most cases, the IterILU(1,3) preconditioner yields similar counts with the exact ILU(0). The iteration counts decrease with increasing p . The IterILU(p, m) preconditioners do not work for the last two test cases.

	row/column	nnz	$p = 1$	$p = 2$	$p = 3$
apache1	80800	542184	1	1.70	3.01
apache2	715176	4817870	1	1.71	3.02
thermal1	82654	574458	1	1.61	2.39
thermal2	1228045	8580313	1	1.62	2.42
parabolic_fem	525825	3674625	1	1.65	2.59
G3_circuit	1585478	7660826	1	1.31	1.69
thermomech_dM	204316	1423116	1	1.68	2.77
thermomech_TC	102158	711558	1	1.68	2.77
ecology2	999999	4995991	1	1.33	1.67
offshore	259789	4242673	1	2.21	6.16
af_shell3	504855	17562051	1	1.42	1.97

Table 5.2:: The second and third columns in this table are the rows and numbers of nonzeros of some SPD matrices from the University of Florida sparse matrix collection [15], respectively. The rightmost two columns are the ratios of nonzeros of the IterILU(p, m) lower factor L with different p and the nonzeros with $p = 1$.

5.2. IterILUT(τ, k)

We test the threshold type factorization IterILUT(τ, p) in Algorithm 3.4. As discussed in the previous section, the parameters of IterILUT(τ, p) should be determined carefully. Here, we consider the *parabolic_fem* matrix in [15] and choose $\tau = 0.025$ and $\tau = 0.006$. The reason we choose these two parameters is that the factors generated by them have similar numbers of nonzeros compared with IterILU($2, m$) and IterILU($3, m$), respectively. Figure 5.4 (left) shows the numbers of nonzeros in the lower factor L of IterILUT(τ, p) as a function of the enhancement number p . While the storage required is similar, we can observe the advantage of the threshold ILU preconditioners. Here, we set the stopping tolerance to 10^{-10} . Figure 5.4 (right) shows the PCG iteration counts as a function of the enhancement number p . Both the numbers of nonzeros in the factors and the PCG iteration counts tend to flatten out after $p = 5$ for $\tau = 0.025$ and $p = 10$ for $\tau = 0.006$. Then we choose these values of p for each τ and we plot the PCG convergence history with different preconditioners in Figure 5.5. These results indicate that while the number of nonzeros is similar, IterILUT(τ, p) performs better than IterILU(p, m) for this problem.

6. Numerical experiments II: eigenvalue problems

In this section, we apply the IterILU(p, m) preconditioner in Algorithm 3.3 to solve eigenvalue problems. We use the Locally Optimal Block Preconditioned Conjugate Gradient method (LOBPCG) [17, 25, 34] to compute part of the spectrum. In addition to preconditioners, there are many factors that impact the convergence behaviors of eigensolvers, such as the initial data, stopping criterion, the number of eigenvalue computed simultaneously and random errors. When the eigensolver convergence degenerates or even fails, the eigensolver can be restarted with different initial data and parameters which may lead to better convergence.

We start by computing some of the smallest eigenvalues of the 3D finite difference Laplacian matrix with 10^6 rows considered in the previous section. We compute the four smallest eigenvalues simultaneously, set the stopping criterion to 10^{-12} and use random initial data.

	no p.c.	exact ILU(0)	IterILU(1,3)	IterILU(2,3)	IterILU(3,3)
apache1	3774	364	359	281	249
apache2	5468	874	874	592	473
thermal1	1734	659	659	369	252
thermal2	6727	2598	2599	1443	984
parabolic_fem	3495	1432	1432	853	553
G3_circuit	19975	1152	1178	667	550
thermomech_dM	89	10	10	6	4
thermomech_TC	89	10	10	6	4
ecology2	7154	2125	2127	1338	1085
	no p.c.	level-0 ILU	IterILU(1,10)	IterILU(2,10)	IterILU(3,10)
offshore	-	567	574	-	-
af_shell3	4700	1172	-	-	-

Table 5.3:: PCG iteration counts of different ILU preconditioners for linear systems from the University of Florida sparse matrix collection [15].

Figure 6.1 shows the LOBPCG convergence history with different preconditioners. The results are similar to solving linear systems with PCG. The level-0 iterative preconditioner has almost the same performance of the exact ILU(0) preconditioner. The convergence rates improve with increasing levels of fill-ins.

6.1. An eigenvalue problem in mixed form

In this subsection, we compute the Laplace eigenvalue problem in mixed form, i.e. we consider a generalized eigenvalue problem generated by mixed finite element discretizations. To this end, we rewrite the Laplace eigenvalue problem

$$\begin{cases} -\Delta u = \lambda u & \text{in } \Omega \\ u = 0 & \text{on } \partial\Omega \end{cases}$$

as a first-order system:

$$\begin{cases} \sigma - \text{grad} u = 0 & \text{in } \Omega \\ \text{div} \sigma = -\lambda u & \text{in } \Omega \\ u = 0 & \text{on } \partial\Omega. \end{cases}$$

We then consider the two-dimensional case and approximate this differential system with Raviat-Thomas elements; see [9–11] and the references therein for more details. The resulting algebraic system can be written in block matrix form as

$$\begin{pmatrix} A & B^T \\ B & 0 \end{pmatrix} \begin{pmatrix} \sigma \\ u \end{pmatrix} = \mu \begin{pmatrix} 0 & 0 \\ 0 & M \end{pmatrix} \begin{pmatrix} \sigma \\ u \end{pmatrix}, \quad (6.1)$$

$$\lambda = -\mu,$$

where A and M are Hermitian matrices.

The standard LOBPCG method finds the eigenvalues at the two ends of the spectrum of Hermitian systems by using a Rayleigh-Ritz procedure. However, the spectrum of (6.1) distributes as follows

$$\mu_M \leq \dots \leq \mu_2 \leq \mu_1 (\leq 0) \leq \infty_1 = \infty_2 = \dots = \infty_N.$$

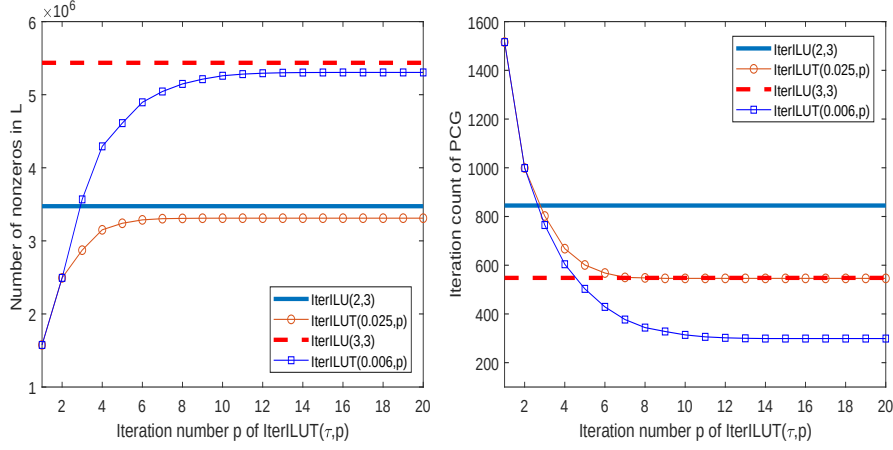


Fig. 5.4.: Left: the number of nonzeros in the lower factor L generated by $\text{IterILUT}(\tau, p)$ with different threshold parameters and iteration numbers. Right: PCG iteration numbers using the $\text{IterILUT}(\tau, p)$ preconditioners with different τ and p .

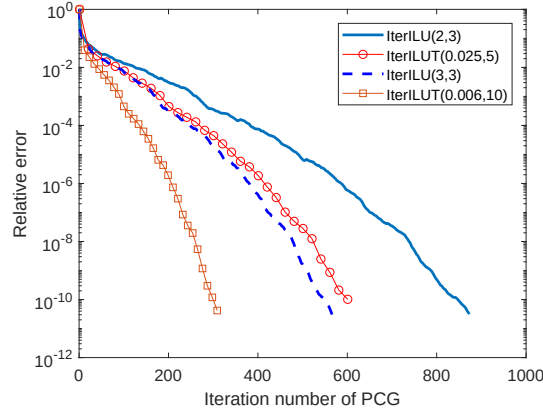


Fig. 5.5.: PCG convergence history with $\text{IterILUT}(\tau, p)$ and $\text{IterILU}(p, m)$ preconditioners for the *parabolic_fem* problem.

Here, the infinite eigenvalues are generated by the eigenpairs $(\sigma, u)^T$ with $\sigma \neq 0$ and $u = 0$. The desired eigenvalues are the ones with smallest magnitude values, which are interior eigenvalues. In order to approximate these eigenvalues, we replace the Rayleigh-Ritz procedure by a Harmonic Rayleigh-Ritz procedure in LOBPCG. Another problem is that the bottom left block in the left-hand side is zero, while the new algorithm generating our preconditioners requires that the diagonal entries are all nonzeros. We then use a spectral transform method to add a mass matrix at both sides of (6.1), shown in Figure 6.2, and obtain an equivalent generalized eigenvalue problem:

$$\begin{pmatrix} A & B^T \\ B & M \end{pmatrix} \begin{pmatrix} \sigma \\ u \end{pmatrix} = \mu \begin{pmatrix} 0 & 0 \\ 0 & M \end{pmatrix} \begin{pmatrix} \sigma \\ u \end{pmatrix}, \quad (6.2)$$

$$\lambda = -\mu + 1.$$

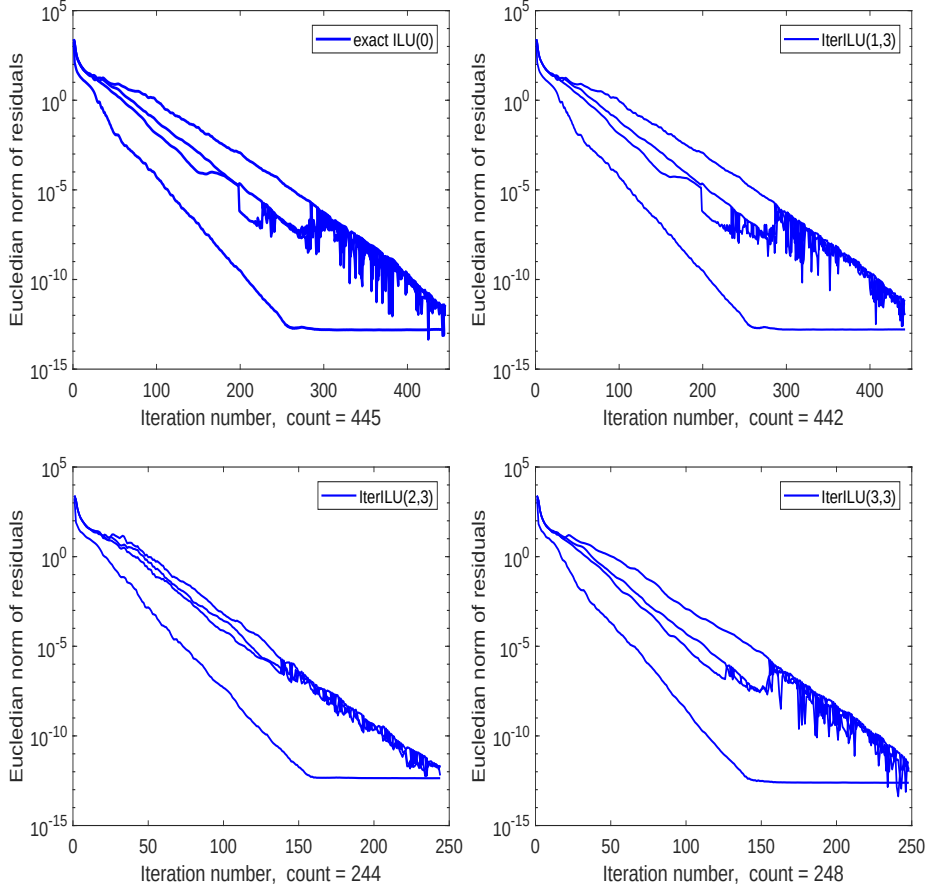


Fig. 6.1.: LOBPCG convergence history in solving the Laplace eigenvalue problem with 10^6 rows using different ILU preconditioners.

Then we can generate preconditioners for the the left-hand side matrix by using the $\text{IterLU}(p,m)$ preconditioner in Algorithm 3.3.

In this test example, the block matrices are generated by the first order RT elements on a square domain $[0, \pi]^2$ with a mesh 128×128 , and have 148225 rows. We set 10^{-12} as the stopping criterion tolerance and compute the six smallest magnitude eigenvalues. The results reported in Figure 6.3 indicate that the $\text{IterLU}(1,3)$ preconditioner has a similar performance as the standard ILU preconditioner. Again, the convergence rates improve with increasing levels of fill-ins.

7. Conclusions and limitations

In this paper, we derived an iterative matrix form of the conventional LU factorization and constructed and analyzed several iterative ILU preconditioners based on this iterative procedure. The new preconditioners have been successfully applied to solving both linear systems and eigenvalue problems. As the main task of the algorithms is a matrix-matrix multiplication, the proposed preconditioners can benefit from fine-grained parallelism. Next, we plan to study the efficient implementation of these algorithms on parallel computing platforms, especially on

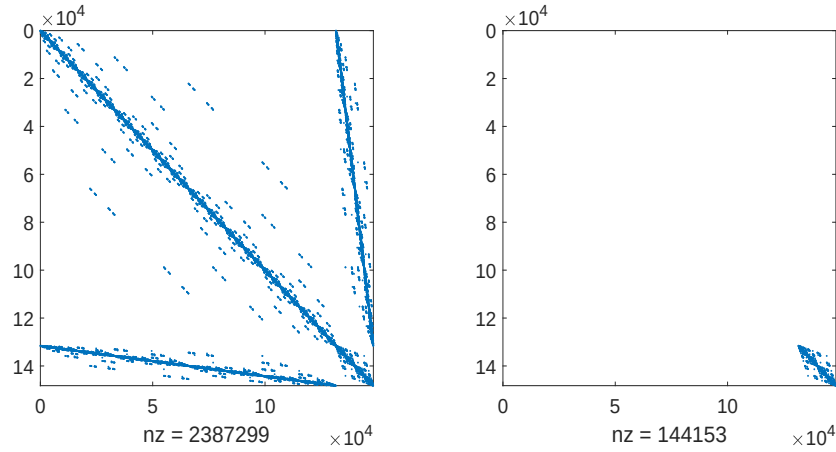


Fig. 6.2.: The sparsity patterns of the matrices generated by mixed FEM.

GPUs, where our iterative ILU preconditioners could yield fast parallel solvers and eigensolvers.

Limitations. The proposed algorithms, especially with higher levels of fill-in, may become unstable and fail for some matrices, as shown in Table 5.3. As the algorithms involve divisions in each iteration, we choose the matrices without zero diagonal entries and assume that there is no breakdown during the computation. Future work is needed in order to deal with these singular cases and to enhance the robustness of the iterative factorizations. Finally, we only present two dropping strategies based on positions and magnitudes, respectively, but it would be interesting to explore hybrid strategies combining positions, magnitudes and numbers of nonzeros together.

A. IterILU code in Matlab

```
function [L,U] = IterILU(A,p,m)
n = size(A,1);
L = sparse(n); U = L;
D = sparse(n,1);

for k = 1 : p
    B = L * U;
    B = A - B;
    D = diag(B); U = triu(B,1); L = tril(B,-1);
    L = L * diag(D.^(-1));
end

SP = (B~=0); % obtain the sparsity pattern SP.

for k = 1 : m
    B = L * U;
    B = A - B;
```

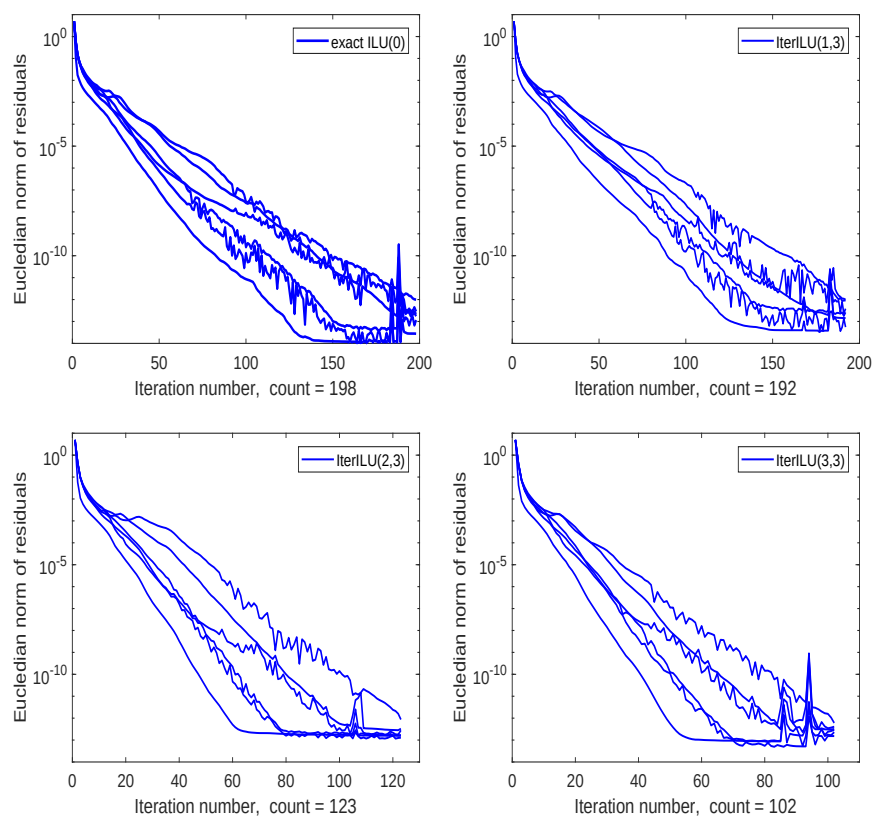


Fig. 6.3.: The convergence history in solving mixed eigenvalue problem with different ILU preconditioners.

```

% delete the entries out of the sparsity pattern SP.
B = B.* SP;
D = diag(B); U = triu(B,1); L = tril(B,-1);
L = L * diag(D.^(-1));
end
L = L + speye(n);
U = U + diag(D);

end

```

Acknowledgments. The authors are members of the INdAM Research group GNCS and their research is partially supported by IMATI/CNR, by PRIN/MIUR and the “Dipartimenti di Eccellenza Program 2018-22 - Dept. of Mathematics, University of Pavia”.

References

- [1] A. Abdelfattah, H. Anzt, J. Dongarra, M. Gates, A. Haidar, J. Kurzak, P. Luszczek, S. Tomov, I. Yamazaki, and A. YarKhan. Linear algebra software for large-scale accelerated multicore computing. *Acta Numerica*, 25:1–160, 2016.

- [2] E. Anderson and Y. Saad. Solving sparse triangular linear systems on parallel computers. *Internat. J. High Speed Comput.*, 1(01):73–95, 1989.
- [3] H. Anzt, E. Chow, and J. Dongarra. ParILUT—a new parallel threshold ILU factorization. *SIAM J. Sci. Comput.*, 40(4):C503–C519, 2018.
- [4] C. C. Ashcraft and R. G. Grimes. On vectorizing incomplete factorization and ssor preconditioners. *SIAM J. Sci. Statist. Comput.*, 9(1):122–151, 1988.
- [5] S. Balay, S. Abhyankar, M. Adams, J. Brown, P. Brune, K. Buschelman, L. Dalcin, V. Eijkhout, W. Gropp, D. Kaushik, et al. *Petsc users manual revision 3.8*. Technical report, Argonne National Lab.(ANL), Argonne, IL (United States), 2017.
- [6] R. Barrett, M. W. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. van der Vorst. *Templates for the solution of linear systems: building blocks for iterative methods*, volume 43. SIAM, 1994.
- [7] M. Benzi. Preconditioning techniques for large linear systems: a survey. *J. Comput. Phys.*, 182(2):418–477, 2002.
- [8] M. Bern, J. R. Gilbert, B. Hendrickson, N. Nguyen, and S. Toledo. Support-graph preconditioners. *SIAM J. Matrix Anal. Appl.*, 27(4):930–951, 2006.
- [9] D. Boffi. Finite element approximation of eigenvalue problems. *Acta Numer.*, 19:1–120, 2010.
- [10] D. Boffi, F. Brezzi, and M. Fortin. *Mixed finite element methods and applications*, volume 44. Springer, 2013.
- [11] D. Boffi, D. Gallistl, F. Gardini, and L. Gastaldi. Optimal convergence of adaptive fem for eigenvalue clusters in mixed form. *Math. Comp.*, 86(307):2213–2237, 2017.
- [12] A. Buluç and J. R. Gilbert. Parallel sparse matrix-matrix multiplication and indexing: Implementation and experiments. *SIAM J. Sci. Comput.*, 34(4):C170–C191, 2012.
- [13] C. Calgario, J.-P. Chehab, and Y. Saad. Incremental incomplete LU factorizations with applications. *Numer. Linear Algebra Appl.*, 17(5):811–837, 2010.
- [14] E. Chow and A. Patel. Fine-grained parallel incomplete LU factorization. *SIAM J. Sci. Comput.*, 37(2):C169–C193, 2015.
- [15] T. A. Davis and Y. Hu. The University of Florida sparse matrix collection. *ACM T. Math. Software (TOMS)*, 38(1):1, 2011.
- [16] H. C. Elman. A stability analysis of incomplete LU factorizations. *Math. Comp.*, pages 191–217, 1986.
- [17] L. Ghezzi, L. F. Pavarino, and E. Zampieri. Overlapping schwarz preconditioned eigensolvers for spectral element discretizations. *Appl. Math. Comput.*, 218(15):7700–7710, 2012.
- [18] J. R. Gilbert, C. Moler, and R. Schreiber. Sparse matrices in matlab: Design and implementation. *SIAM J. Matrix Anal. Appl.*, 13(1):333–356, 1992.
- [19] A. Greenbaum. *Iterative methods for solving linear systems*, volume 17. SIAM, 1997.
- [20] I. Gustafsson. A class of first order factorization methods. *BIT*, 18(2):142–156, 1978.
- [21] P. Hénon and Y. Saad. A parallel multistage ILU factorization based on a hierarchical graph decomposition. *SIAM J. Sci. Comput.*, 28(6):2266–2293, 2006.
- [22] D. Hysom and A. Pothen. A scalable parallel algorithm for incomplete factor preconditioning. *SIAM J. Sci. Comput.*, 22(6):2194–2215, 2001.
- [23] D. Hysom and A. Pothen. Level-based incomplete LU factorization: Graph model and algorithms. *Preprint UCRL-JC-150789, US Department of Energy, Nov*, 2002.
- [24] D. S. Kershaw. The incomplete Cholesky conjugate gradient method for the iterative solution of systems of linear equations. *J. Comput. Phys.*, 26(1):43–65, 1978.
- [25] A. V. Knyazev. Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method. *SIAM J. Sci. Comput.*, 23(2):517–541, 2001.
- [26] X. S. Li. An overview of SuperLU: algorithms, implementation, and user interface. *ACM T. Math. Software (TOMS)*, 31(3):302–325, 2005.
- [27] S. MacLachlan, D. Osei-Kuffuor, and Y. Saad. Modification and compensation strategies for

- threshold-based incomplete factorizations. *SIAM J. Sci. Comput.*, 34(1):A48–A75, 2012.
- [28] M. Magolu monga Made and H. A. van der Vorst. A generalized domain decomposition paradigm for parallel incomplete LU factorization preconditionings. *Future Gener. Comp. Sy.*, 17(8):925–932, 2001.
 - [29] M. McCourt, B. Smith, and H. Zhang. Sparse matrix-matrix products executed through coloring. *SIAM J. Matrix Anal. Appl.*, 36(1):90–109, 2015.
 - [30] Y. Notay. Conditioning analysis of modified block incomplete factorizations. *Linear Algebra Appl.*, 154:711–722, 1991.
 - [31] Y. Saad. ILUT: A dual threshold incomplete LU factorization. *Numer. Linear Algebra Appl.*, 1(4):387–402, 1994.
 - [32] Y. Saad. *Iterative methods for sparse linear systems*, volume 82. SIAM, 2003.
 - [33] J. Scott and M. Tuma. Improving the stability and robustness of incomplete symmetric indefinite factorization preconditioners. *Numer. Linear Algebra Appl.*, 24(5):e2099, 2017.
 - [34] E. Vecharynski and A. V. Knyazev. Preconditioned locally harmonic residual method for computing interior eigenpairs of certain classes of hermitian matrices. *SIAM J. Sci. Comput.*, 37(5):S3–S29, 2015.
 - [35] A. J. Wathen. Preconditioning. *Acta Numer.*, 24:329–376, 2015.